

Relatório Executivo de Arquitetura

Projeto: Estudo de Arquitetura de Sistemas Inteligentes (Padrão PycoderBR / SCSi)

Módulo: Fase 7 — Mensageria Assíncrona & IA Cognitiva Local

Data de Homologação: 24/09/2026

Status: 100% Homologado por Bancada de Especialistas

Este relatório formaliza a conclusão e aprovação arquitetural dos componentes responsáveis pelo desacoplamento de tarefas assíncronas e integração soberana de Inteligência Artificial Generativa no cluster Docker Swarm.

1. RabbitMQ (Passo 12) - O Broker de Mensagens

- **Imagem e Perfil:** `rabbitmq:3-management-alpine` (Acesso à API de métricas REST para Antigravity via ADR 011).
- **Segurança (Zero Trust):** Uso de credenciais em RAM via Docker Secrets (`RABBITMQ_DEFAULT_USER_FILE`).
- **Soberania de Rede:** Ancorado estritamente à rede overlay `scsi_internal`.
- **Hotfixes Aplicados:** Inclusão de healthcheck de liveness para prevenir *crash loops* nos workers. Uso do padrão "Payload Magro" (Ids em vez de dados densos) respeitando teto de 1GB de RAM.

2. Celery Worker (Passo 13) - Operários Assíncronos

- **Princípio DRY:** Reutilização integral da imagem Multi-stage do Django (compartilhando ORM e infraestrutura de rede).
- **I/O Bound & Threads:** Padrão de processo alterado para `--pool=threads --concurrency=10`, evitando o alto custo de RAM do **Prefork** e garantindo máxima vazão para chamadas de rede da IA. Veto ao *gevent* aplicado para evitar **deadlocks** com bibliotecas C/gRPC (LangChain).

- **Fencing & Routing:** Teto cravado em 1024M de RAM. Fila isolada -q ia_tasks para não concorrer com tarefas *CPU Bound* ou de sistema. Liveness probe injetado.

3. Celery Beat (Passo 14) - Orquestrador Temporal

- **Singleton Padrão-Ouro:** Implantação restrita a replicas: 1 e pinada no manager, blindando o ecossistema contra a duplicação de tarefas agendadas (Split-Brain).
- **DatabaseScheduler:** Tarefas (como limpezas de sessão e manutenção programada) cadastradas dinamicamente via painel Django (PostgreSQL) sem necessidade de rebuild.
- **Ressalvas Resolvidas:** Ajuste do timezone `CELERY_ENABLE_UTC=False` e abstenção consciente de healthchecks nativos, que induziriam o container ao erro.

"O paradigma adotado eleva a plataforma à classificação máxima de Soberania de Dados (Zero-Leak), garantindo que PII e inteligência semântica jamais transitem por APIs de terceiros como OpenAI ou Anthropic."

4. Ollama & LangGraph (Passo 15) - IA Soberana

- **Motor de Inferência (Ollama):** Executando nativamente no Swarm (`scsi_internal`) com Memory Fencing agressivo (Limite de 4GB de RAM). Adoção da flag vital `OLLAMA_KEEP_ALIVE=60s` para purgar a RAM do host pós-processamento.
- **Stack Cognitiva (Sub-4B):** Geração de texto pelo `llama3.2` (ou `qwen2.5:3b`) provendo velocidade na CPU e ampla sobra de RAM (~2GB) para janelas de contexto longo (*KV Cache* do RAG). Vetorização via `nomic-embed-text`.
- **Orquestração:** Fluxos de pensamento e *state graphs* gerenciados pelo LangGraph rodando inteiramente na camada assíncrona do Celery Worker, consultando vetores semânticos diretamente no pgvector (PostgreSQL 16).